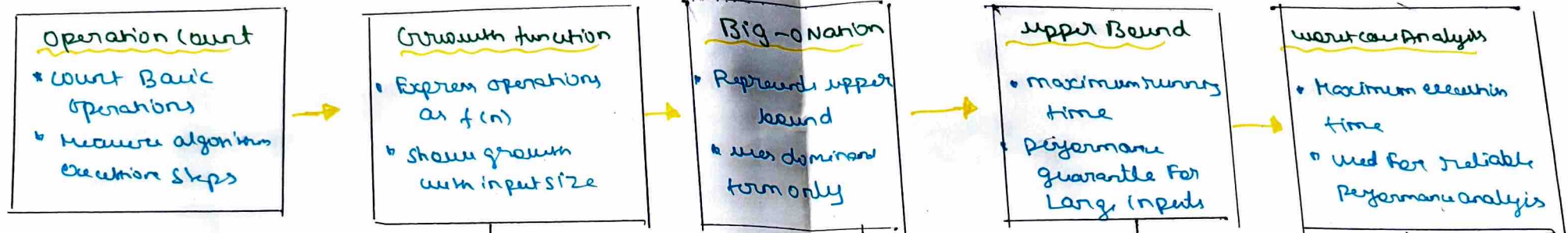


Q) 35

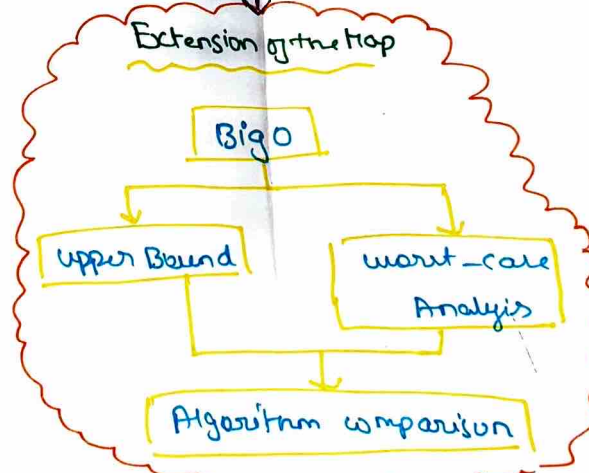
Operation Growth $\rightarrow$ Growth Function $\rightarrow$ Big-O $\rightarrow$ Upper Bound $\rightarrow$ Worst-case Analysis



How it supports Algorithm comparison?

- compares algorithms
- predicts performance
- Best algorithm
- Independent of hardware

Extension of the Map



Example:

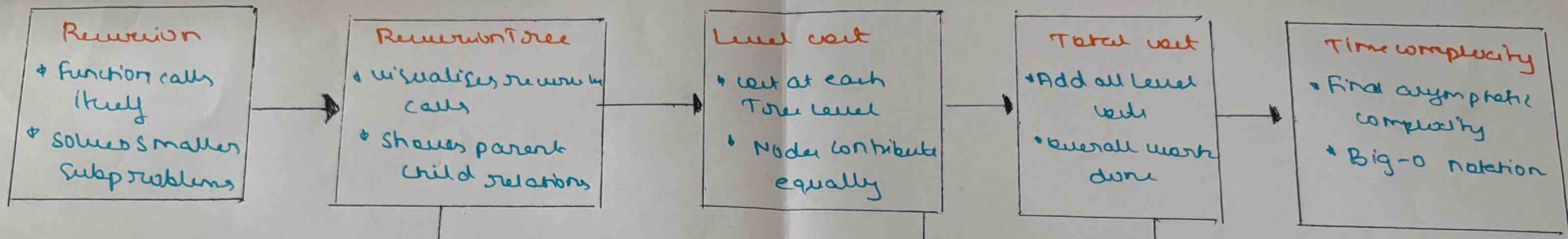
- Algorithm: Linear search
- Operation count: n comparisons
- Growth function: $f(n) = n$
- Big-O: $O(n)$
- Upper Bound: $O(n)$
- Worst case: Element found at last position

Justification: Why Big-O Represents worst-case Behaviour?

- Big-O gives an upper bound on the running time
- It guarantees that the time will not exceed this bound for any input
- Therefore, it represents the worst-case behaviour of an Algorithm.

75

Recursion $\rightarrow$ Recursion Tree $\rightarrow$ Level cost $\rightarrow$ Total cost $\rightarrow$ Time complexity



Tree Height

- No. of levels
- Depends on recursion depth

Branching factor

- Children per node
- Affects total nodes

How it Simplifies Analysis?

- Visualises recursion
- Easy cost calculation
- Compare recursive algorithms
- Predict performance

Summing Level costs

- Add cost of every level
- Gives total running time

Example

- Merge Sort
- Binary Search
- $T(n) = 4T(n/2) + n$

Justification: Why Summing Level Cost gives final complexity?

- Each level represents work done at that stage of recursion
- Total work = Sum of work at all levels
- As input size grows, this sum dominates the running time